WEEK 3 · LESSON 13 OF 24 · HANDOUT

Embeddings: Meaning as Numbers

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=Q_slEpNuhMc — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2024: Deconstructing the text embedding models — https://www.youtube.com/watch?v=Lha_QCkXxm0

🎓 **Go deeper:** PyCon Italia: Word Embeddings for Natural Language Processing in Python — https://www.youtube.com/watch?v=c8QF1FJjPG8

Your goal in this lesson: Explain what text embeddings are, compare texts with cosine similarity, build simple embeddings yourself, and know when to use a modern embedding model.

Turning meaning into a list of numbers

An **embedding** is a vector — a list of numbers — that represents the meaning of a piece of text. Texts with similar meaning get vectors that point in similar directions, even if they use different words:

- "How much does membership cost?" and "What is the monthly fee?" → close vectors.
- "How much does membership cost?" and "Where are the toilets?" → far apart.

Embeddings power **semantic search**, recommendations, clustering of feedback, duplicate detection and — most importantly for this week — **RAG**.

Cosine similarity

To compare two vectors, measure the angle between them:

> *cosine similarity = (a · b) ÷ (|a| × |b|) — 1 means the same direction, 0 means unrelated*

```
import numpy as np

def cosine(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

# toy 3-number "embeddings": [money, technology, food]
fee = np.array([0.9, 0.1, 0.0])
price = np.array([0.8, 0.2, 0.1])
wifi = np.array([0.1, 0.9, 0.0])
print("fee vs price:", round(cosine(fee, price), 2))
print("fee vs wifi: ", round(cosine(fee, wifi), 2))
```

"fee" and "price" are about **0.98** similar; "fee" and "wifi" only about **0.22**. Real embeddings have hundreds or thousands of numbers, learned by neural networks — no human chooses what each number means.

Our document collection

All week we work with a made-up handbook of a technology hub in Kigali:

```
HANDBOOK = {
    "Opening hours": "The Kigali Tech Hub is open Monday to Friday from 8:00 to 20:00 and on
Saturday "
                     "from 9:00 to 14:00. The hub is closed on Sundays and on public
holidays.",
    "Membership plans": "There are three plans. The Student plan costs 10,000 RWF per month and
needs a "
                        "valid student card. The Standard plan costs 25,000 RWF per month. The
Team plan "
                        "costs 90,000 RWF per month for up to five people.",
    "Joining": "To become a member, fill in the online form, bring your national ID or passport
to the "
               "front desk, and pay the first month. Your access card is ready the same day.",
    "Payment": "Fees are paid by mobile money or bank transfer before the 5th of each month.
Members who "
               "pay late for two months lose access until the balance is cleared.",
    "Cancelling": "You may cancel your membership at any time with 14 days' written notice by
email. "
                  "Fees already paid for the current month are not refunded.",
    "Wi-Fi": "The Wi-Fi network is called KTH-Members. The password is printed on your access
card and "
             "changes every three months. Guests use the separate KTH-Guest network.",
    "Meeting rooms": "Members can book the two meeting rooms through the hub app. Standard and
Team members "
                     "get 4 free hours per month; Student members get 1 free hour. Extra hours
cost 5,000 RWF.",
    "Guests": "Each member may bring up to two guests per day. Guests must sign in at the front
desk "
              "and leave by 18:00.",
    "Printing": "Black-and-white printing costs 50 RWF per page and colour printing 200 RWF per
page. "
                "Students get 100 free black-and-white pages per month.",
    "Events": "The hub runs a free coding workshop every Wednesday at 17:00 and a startup pitch
night "
              "on the last Friday of each month. Members can register in the hub app.",
    "Lockers": "Lockers can be rented for 3,000 RWF per month. Lost locker keys cost 5,000 RWF
to replace.",
    "Code of conduct": "Members must respect others, keep noise low in the quiet zone, and
never share their "
                       "access card. Harassment leads to immediate cancellation of
membership.",
    "Lost and found": "Items left behind are kept at the front desk for 30 days, then
donated.",
    "Accessibility": "The ground floor, meeting rooms and toilets are wheelchair accessible. A
lift serves "
                     "all floors. Contact the front desk to arrange other support.",
}
chunks = [{"id": i, "title": t, "text": f"{t}. {body}"} for i, (t, body) in
enumerate(HANDBOOK.items())]
print(len(chunks), "sections")
```

Keyword search fails on different words

The classic approach, **TF-IDF**, scores texts by shared words, weighting rare words more:

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(stop_words="english")
doc_tfidf = tfidf.fit_transform([c["text"] for c in chunks])

def keyword_search(query, k=3):
    scores = (doc_tfidf @ tfidf.transform([query]).T).toarray().ravel()
    best = np.argsort(scores)[::-1][:k]
    return [(chunks[i]["title"], round(float(scores[i]), 2)) for i in best]

print(keyword_search("membership plans price"))
print(keyword_search("If I quit, do I get my money back?"))
```

The first query shares words with the "Membership plans" section and works. The second shares almost **no words** with the "Cancelling" section ("quit" vs "cancel", "money back" vs "refunded"), so keyword search struggles — the right section is missing or ranked low.

Embeddings capture related words

Latent semantic analysis compresses TF-IDF into a few dimensions with SVD (the PCA idea from Machine Learning Fundamentals), so words that appear in similar contexts move together. It runs locally, with no download:

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import Normalizer

texts = [c["text"] for c in chunks]
embedder = make_pipeline(TfidfVectorizer(stop_words="english", sublinear_tf=True),
                         TruncatedSVD(n_components=12, random_state=0),
                         Normalizer())                      # unit length: dot product = cosine
similarity
doc_vectors = embedder.fit_transform(texts)

def embed(text_list):
    return embedder.transform(text_list)

print("each section is now a vector of", doc_vectors.shape[1], "numbers")
```

```
def semantic_search(query, k=3):
    scores = doc_vectors @ embed([query])[0]
    best = np.argsort(scores)[::-1][:k]
    return [(chunks[i]["title"], round(float(scores[i]), 2)) for i in best]

print(semantic_search("membership plans price"))
print(semantic_search("When can I visit on the weekend?"))
```

The first query works well (Membership plans scores about **0.84**). The second returns scores of **0.0** for every section: "visit" and "weekend" never appear in the handbook, so our embedder has never seen them and turns the question into an empty vector. Our home-made embeddings only know the words in 14 short sections. **Modern embedding models** are neural networks trained on billions of sentences, and they handle synonyms, paraphrases and even several languages far better.

Modern embedding models

Option	Notes
Open models with `sentence-transformers` (for example multilingual MiniLM, E5, BGE)	Free, run in Colab or on your own server; some support Kinyarwanda and Swahili reasonably well
Embedding APIs (Voyage AI, OpenAI, Google, Cohere)	Paid per token, very good quality, no GPU needed

The code shape is always the same: `vectors = model.encode(list_of_texts)` (or an API call), then cosine similarity. In the rest of the week, swap our `embed()` function for a real model in your own projects — nothing else changes.

Key terms

- **Embedding** — a vector that represents meaning.
- **Cosine similarity** — how closely two vectors point the same way (−1 to 1).
- **Semantic search** — search by meaning instead of exact words.
- **TF-IDF** — word-count scores that weight rare words more.
- **Embedding model** — a neural network that turns text into embeddings.

Remember this: Embeddings turn meaning into vectors so we can search by meaning; keyword search misses paraphrases, and modern embedding models handle them far better than simple methods.

Name: _____ Date: _____

Activity

Task 1

Compute the cosine similarity of `fee` with itself and with `-fee`. What do 1 and −1 mean?

Task 2

Try five questions of your own with `keyword_search` and `semantic_search`. When does each win?

Task 3

Change `n_components` to 4 and to 13. How do the results change?

Task 4

Group the support messages from Week 2 into clusters with k-means on their embeddings (Machine Learning Fundamentals, Lesson 19).

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is a text embedding?
   - A. A compressed file
   - B. A tokenizer
   - C. A database
   - D. A vector of numbers representing the meaning of text
2. Cosine similarity of 1 means…
   - A. Opposite
   - B. Missing data
   - C. Pointing in exactly the same direction
   - D. Unrelated
3. Why does keyword search fail for 'If I quit, do I get my money back?'
   - A. It is too long
   - B. TF-IDF cannot read questions
   - C. It shares almost no words with the 'Cancelling' section
   - D. The section is missing
4. What does semantic search compare?
   - A. Dates
   - B. Exact words
   - C. File sizes
   - D. Meaning, through embeddings
5. Why are modern embedding models much better than our home-made ones?
   - A. They only use keywords
   - B. They are neural networks trained on billions of sentences
   - C. They use more colours
   - D. They need no input
6. Which is an open-source way to create embeddings?
   - A. A spreadsheet
   - B. Printing the text
   - C. sentence-transformers models
   - D. A chat app

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.